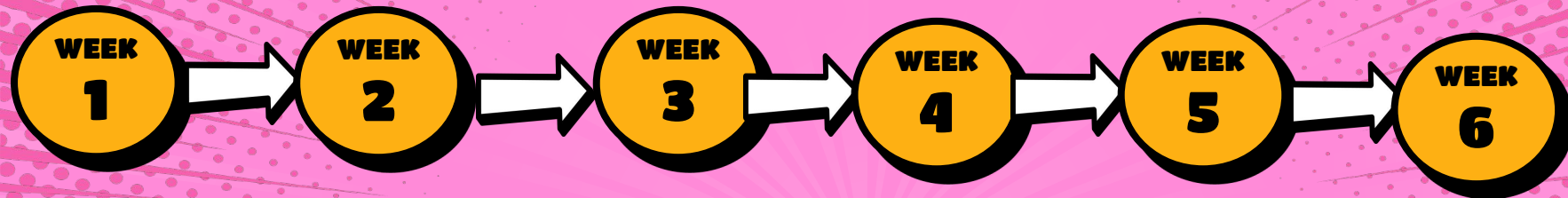




summarizer !!

23/02/2023

Pedro Jerónimo LinkedList ??????



How Pc Works

Start Java

Nice Java

Fck Java

Game

Ohhh! no !!! more
Java

"Hurry up and live"



World Wide Web

WWW

"We suffer more often in imagination than in reality"



www fundamental technologies

01. HTTP

- HyperText Transfer Protocol
- The HTTP protocol is a stateless one. This means that every HTTP request the server receives is independent and does not relate to requests that came prior to it
- HTTP is connectionless: The HTTP client, i.e., a browser initiates an HTTP request and after a request is made, the client waits for the response.

02. URI

- A Uniform Resource Identifier (URI) is a unique sequence of characters that identifies a logical or physical resource used by web technologies.
- The real difference between a URI and a URL is that a URI can be just a name by itself (like google.com) or a name combined with a protocol for how to get there (like https://)

03. HTML

- HTML is the standard markup language for Web pages
- With HTML you can create your own Website
- HTML is easy to learn – You will enjoy it!

04. What ??? is this

- *"I am nothing.*
 - *I'll never be anything.*
 - *I couldn't want to be something.*
 - *Apart from that, I have in me all the dreams in the world."*
- 



HTTP Verbs(most popular)

01.

GET

The GET method requests a representation of the specified resource. Requests using GET should only retrieve data.



02.

POST

The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.



03.

PUT

The PUT method replaces all current representations of the target resource with the request payload.

04.

DELETE

The DELETE method deletes the specified resource.



"Ignorance is the cause of fear."

HTTP Status Codes

Level 400

400 : Bad Request
401 : Unauthorized
403 : Forbidden
404 : Not Found
409 : Conflict

Level

500 :

503 : S

501 : M

504 :

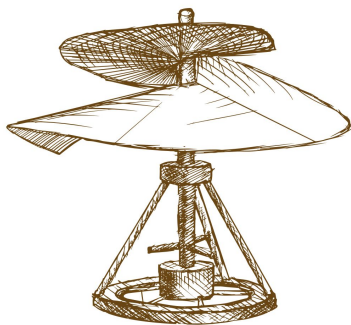
599

50

HTTP Status Codes

With URLs and verbs, the client can initiate requests to the server. In return, the server responds with status codes and message payloads. The status code is important and tells the client how to interpret the server response

**"Let everything happen to you: beauty and terror.
Just keep going. No feeling is final."**



java.net.URL

Immutable object representing an abstraction of a Uniform Resource Locator



advantages

Comes with the capability of retrieving the URL contents and support for proxy servers and multiple protocols, making use of the Strategy design pattern



fantastic socks!!

java.net.URLConnection and its subclasses provide advanced features and control. They provide a way to use sockets in your code without having to know any of the underlying socket details



NAPOLETANA PASTA

"He who is brave is free."

**{ Exercise }
Simple Web
Server**





Our Goals

Goal 01.

Server response text/html

Goal 02.

Server response image

Goal 03.

Server response 404 Not Found

Java Magic

1

ServerSocket

2

Socket

3

BufferedReader

4

BufferedWriter

5

InputStreamReader

6

OutputStreamWriter

```
public static void main(String[] args) {

    int portNumber = 8090;

    try {
        //System.out.println("Server attempting to bind to port " + portNumber);
        ServerSocket socket = new ServerSocket(portNumber);
        //System.out.println("Bind successful! Waiting for connection...");

        Socket clientSocket = socket.accept();
        //System.out.println("Connection established!");

        BufferedReader reader = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
        //System.out.println(reader.readLine());

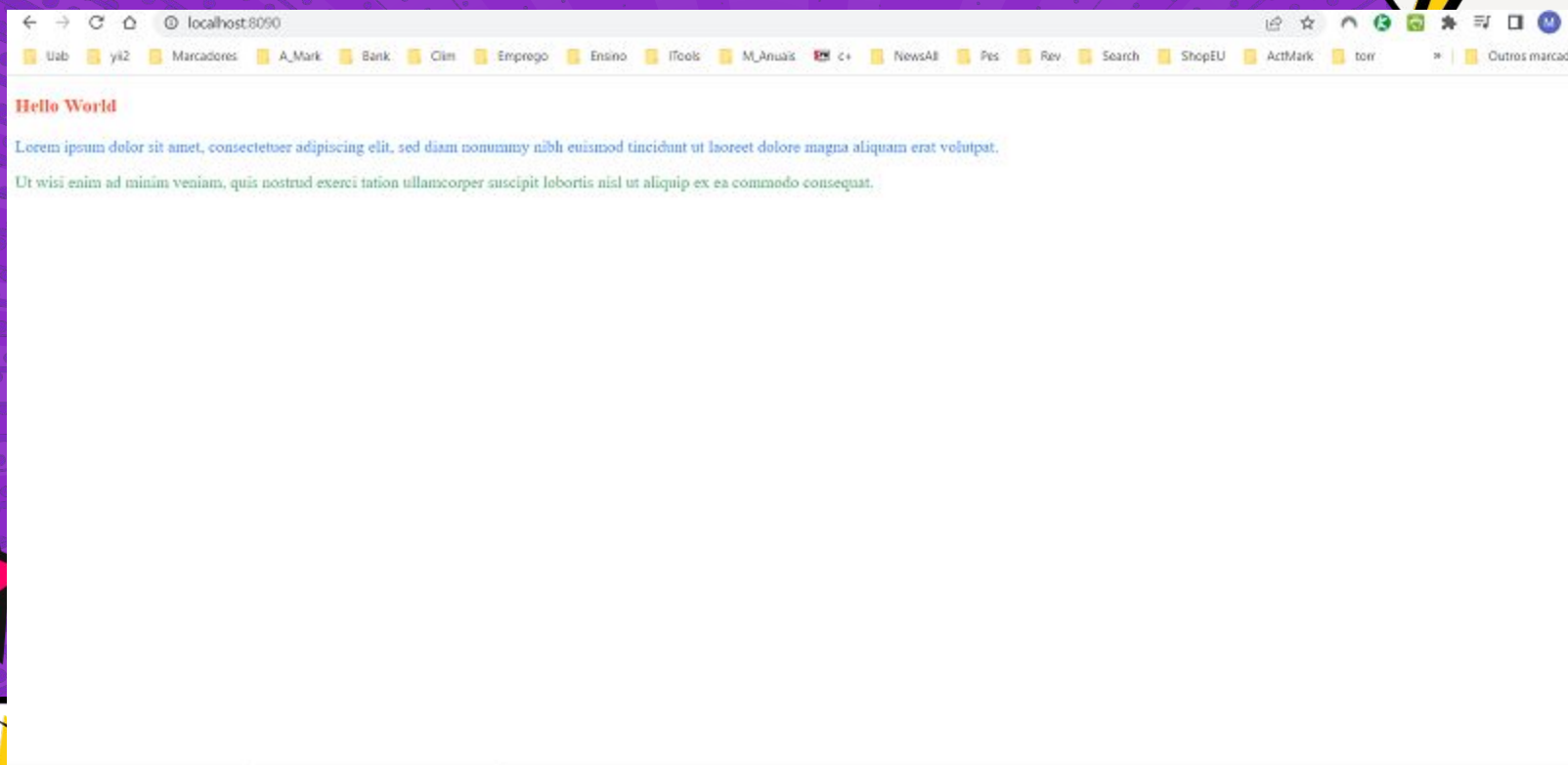
        BufferedWriter writer = new BufferedWriter(new
OutputStreamWriter(clientSocket.getOutputStream()));

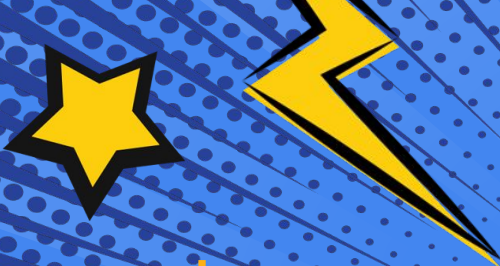
        File file=new File("resources/ola.html");
        BufferedReader fileReader =new BufferedReader(new FileReader(file));
```

Let's see s

```
writer.write("HTTP/1.0 200 Document Follows\r\n" +  
    "Content-Type: text/html; charset=UTF-8\r\n" +  
    "Content-Length: "+file.length()+" \r\n" +  
    "\r\n");  
  
writer.flush();  
  
String temp=fileReader.readLine();  
  
while (temp!=null) {  
    writer.write(temp);  
    writer.flush();  
    temp=fileReader.readLine()    ;  
}  
  
} catch (IOException e) {  
    throw new RuntimeException(e);  
}
```







DONE

**Server response
text/html**

**Server
response image**

**Server response
404 Not Found**

**Not Found :) must make
summarizer**



**You must output bytes !!!ohhh !!
halfway :(**



Thanks !

POW !